MFE FORMWORK MR11 SYSTEM

Deployment Guide

Version 1.0

Prepared for

MFE Formwork

Prepared by

MyIntellX project team

Date

21 October 2026

Status

Draft for client review

Document control

Item	Detail
Document	Deployment Guide
System	MFE Formwork MR11 System, version 1.0
Document version	1.0
Date	21 October 2026
Status	Draft for client review
Based on	Repository izzatmyintellx05/MFE, branch main, commit 0cc8c36
Prepared for	MFE Formwork
Prepared by	MyIntellX project team
Intended readers	Client IT
Classification	Confidential, for client and vendor use only

Revision history

Version	Date	Description	Author
1.0	21 October 2026	First issue for client review	MyIntellX project team

Contents

1. Introduction 4
2. Environments 4
3. What you need before you start 4
4. Environment variables 4
5. Setting up the database in Supabase 5
6. Deploying to Vercel (recommended) 6
7. Deploying to Azure (alternative) 6
7.1 Using the deployment script 6
7.2 Using Docker 6
7.3 Using Azure DevOps 6
8. Running it on your own machine 6
9. Checking the deployment 6
10. Go-live checklist 7
11. Updates and rolling back 7
12. Build and maintenance commands 8

Appendix A: Screenshots 9

1. Introduction

This guide walks the client's IT team through installing the MR11 System and keeping it up to date. We recommend hosting it on Vercel in the Singapore region, with a Supabase PostgreSQL database in `ap-southeast-1`. Azure App Service is a supported alternative.

A first deployment takes about an hour, and most of that time goes into creating the accounts and the database. Once it is running, the checks in section 9 confirm that everything works.

2. Environments

Environment	Hosting	Database	What it is for
Local demonstration	`npm run dev` on a laptop	None; it runs in memory and loads the sample workbooks from `uploads/`	Demonstrations only, because data is lost when it stops
Local with a database	`npm run dev`	A local PostgreSQL on port 5433 (`npm run db:start`)	Development and testing
Staging	A Vercel preview deployment	A separate Supabase project	User acceptance testing and trying out changes
Production (recommended)	Vercel, region `sin1`	Supabase PostgreSQL, through the transaction pooler	Live use
Production (alternative)	Azure App Service (Linux, Node 22) or Docker	Supabase PostgreSQL	When the client prefers Azure

3. What you need before you start

- Node.js 22 and npm. Keep the repository's `.npmrc` file, which contains `legacy-peer-deps=true`.
- Access to the Git repository.
- A Supabase account for the production database.
- A Vercel account linked to the repository, or an Azure subscription with the Azure CLI installed.
- Optionally, PostgreSQL 18 on your own machine, and Docker.

4. Environment variables

Variable	Required?	Example or default	What it does
`DATABASE_URL`	Yes, in production	`postgresql://postgres.<ref>:<password>@aws-0-ap-southeast-1.pooler.supabase.com:6543/postgres?pgbouncer=true`	Connects to Supabase. `DATABASE_POSTGRES_URL` or `POSTGRES_URL` work too. Without it, the system runs in memory only

Variable	Required?	Example or default	What it does
JWT_SECRET	Yes	A random string of at least 32 characters	Signs session tokens. Changing it signs everyone out. If it is not set, the code falls back to a default that is published in the repository, so never let that happen in production
NODE_ENV	Yes, in production	production	Makes the server serve the built frontend
PORT	No	3000	The port used by npm start and Docker
USD_TO_MYR_RATE	No	4.20	A fallback exchange rate, used only if no exchange-rate service can be reached
STORAGE_DIR	No	uploads_storage	The folder for stored files (local and Azure)
WEB_DIR	No	public	The folder holding the built frontend for the API server
DISABLE_HMR	No	true	Turns off live reload during development

Put the real values in Vercel (Settings, then Environment Variables) or in Azure (Configuration, then Application settings). On your own machine, use a `.env` file, which Git ignores. You can generate a good secret with `openssl rand -hex 32`.

5. Setting up the database in Supabase

1. Create a Supabase project in the `ap-southeast-1` (Singapore) region, with a strong database password.
2. Open **SQL Editor** and choose **New query**. Paste in the whole of `supabase/schema.sql` and run it. This creates the tables and adds the nine roles, the seven departments, the administrator and eight demo accounts.
3. Go to **Project Settings**, then **Database**, and copy the **Transaction pooler** connection string (port 6543). This is your `DATABASE_URL`.
4. After you first sign in, change the administrator password straight away, and either delete the demo accounts or give each one a new password.

Please note that `npm run db:seed` only fills the in-memory store and does not write anything to Supabase, so use `supabase/schema.sql` instead.

6. Deploying to Vercel (recommended)

1. In Vercel, choose **Add New**, then **Project**, and import the repository.
2. Leave the build settings as they are. `vercel.json` already sets the build command (`npm run build`), the output folder (`dist`), the `sin1` region and the API routing.
3. Add `DATABASE_URL`, `JWT_SECRET` and `NODE_ENV=production` for Production. If you use preview deployments, add them for Preview too, pointing at the staging database.
4. Deploy. The build produces the frontend in `dist/` and the API bundle in `api/index.js`.
5. Run the checks in section 9.

From then on, every change that reaches `main` is deployed automatically.

7. Deploying to Azure (alternative)

7.1 Using the deployment script

Set `DATABASE_URL`, then run `./deploy-azure.sh`. The script signs in to Azure, creates the resource group `rg-mfe-formwork-prod`, a Linux B1 App Service plan and a Node 22 web app, sets the application settings (including a random `JWT_SECRET`), and deploys a zip of the build. Before you run it, change `LOCATION`: the script uses `eastus` by default, and `southeastasia` is much closer to Malaysia.

7.2 Using Docker

```
docker build -t mfe-mr11 .
docker run -p 3000:3000 -e DATABASE_URL="<supabase url>" -e JWT_SECRET="<secret>" mfe-mr11
```

7.3 Using Azure DevOps

`azure-pipelines.yml` builds the project whenever `main` or `azure-deploy` changes, and deploys only from `azure-deploy`. Before using it, replace the `JWT_SECRET` that is written in the file with a secret pipeline variable, and add `DATABASE_URL` to the application settings.

8. Running it on your own machine

```
git clone https://github.com/izzatmyintellx05/MFE.git
cd MFE
npm install
npm run dev
```

Then open http://localhost:3000. If `DATABASE_URL` is not set, the system loads the sample workbooks in `uploads/` and keeps everything in memory.

9. Checking the deployment

Check	How to do it	What you should see
The application is running	Open `https://<host>/health`	`{"status":"ok"}`
The database is connected	Open `https://<host>/api/health/db`	`"status":"CONNECTED"` and a list of tables

Check	How to do it	What you should see
Signing in works	Sign in as the administrator	The MR11 page opens
Uploading rebuilds the MR11	Upload the BD workbook on the Business Development page	A success message, and the BD rows appear in the MR11
Market data is coming through	Open the MR11 page	The exchange-rate and LME cards show today's values and their sources
Export works	Click Export on the MR11 page	An .xlsx file downloads
Each environment has its own secret	Sign in on production, then try the same token on staging	Staging rejects the production token

See Figure 1 in Appendix A: The sign-in page you should see at https://<host>/ once the deployment is running.

See Figure 2 in Appendix A: After the BD workbook is uploaded, the MR11 lists its rows and the market cards show live values.

10. Go-live checklist

☐ The production `JWT_SECRET` is a random value, different from staging's

☐ `DATABASE_URL` points at the production Supabase project

☐ The administrator password has been changed, and the demo accounts removed or given new passwords

☐ The repository has been made private

☐ `/api/health/db` has been restricted or removed, and CORS limited to the production domain

☐ The custom domain and its HTTPS certificate are working

☐ Backups are confirmed on the Supabase plan

☐ Every department's latest workbook has been uploaded

☐ All the checks in section 9 pass

11. Updates and rolling back

Task	On Vercel	On Azure
Releasing a new version	Merge into `main`	Run the pipeline from `azure-deploy`, or run the script again
Rolling back	Open **Deployments**, pick the last good one and choose **Promote to Production**	Redeploy the previous build package
Changing a setting	Edit the variable, then redeploy	Edit the application settings, and the app restarts by itself

Rolling back the application does not change the database. Changes to the database are rare in this system, but always take a backup before making one.

12. Build and maintenance commands

Command	What it does
`npm run dev`	Starts the development server on port 3000
`npm run build`	Builds the production frontend (`dist/`) and the serverless API bundle (`api/index.js`)
`npm start`	Runs the server, in production mode when `NODE_ENV=production` and `dist/` exists
`npm run lint`	Runs the TypeScript type check
`npm run db:start`, `db:stop` and `db:status`	Control a local PostgreSQL on port 5433
`npm run test:persistence`	Builds the project, then runs the end-to-end persistence test against a local database

Appendix A: Screenshots

These screenshots were taken from version 1.0 running with the sample workbooks, so the projects and figures in them are examples rather than live data.

Figure	What it shows	Mentioned in
Figure 1	The sign-in page you should see at https://<host>/ once the deployment is running	9. Checking the deployment
Figure 2	After the BD workbook is uploaded, the MR11 lists its rows and the market cards show live values	9. Checking the deployment

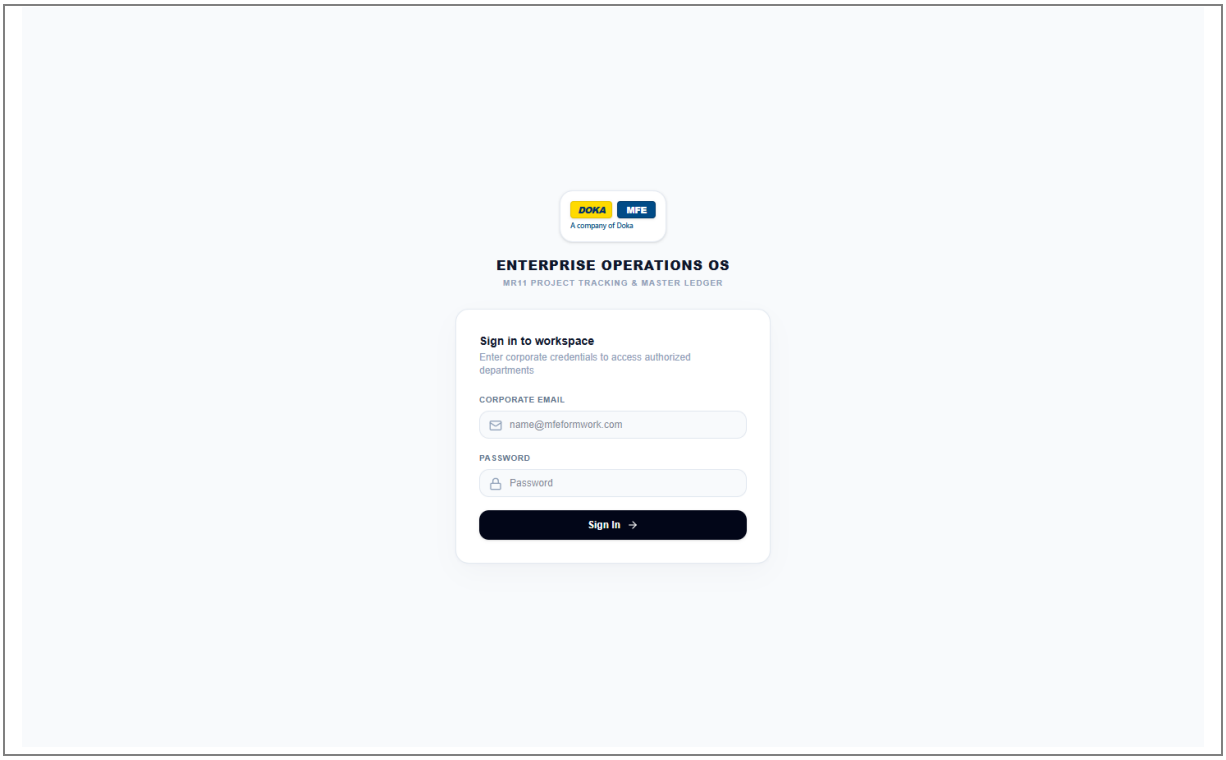


Figure 1: The sign-in page you should see at https://<host>/ once the deployment is running

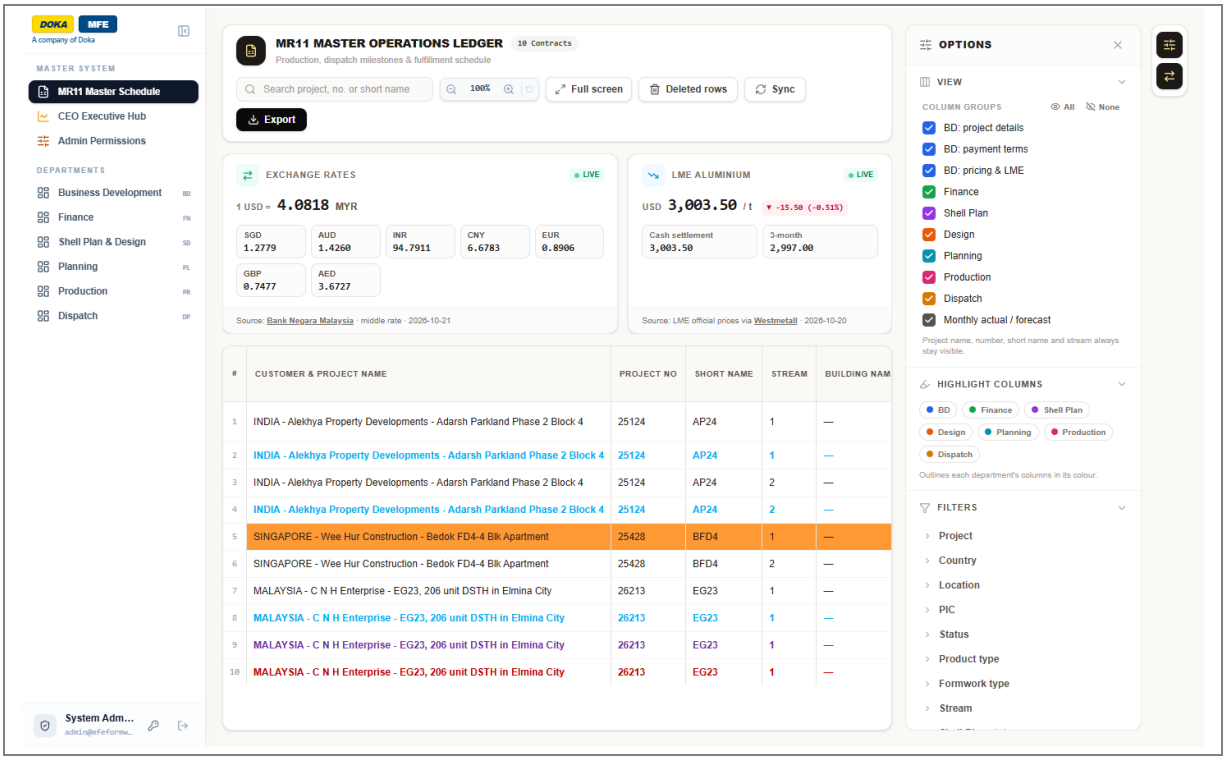


Figure 2: After the BD workbook is uploaded, the MR11 lists its rows and the market cards show live values